

面向跨 OS 底座的可移植 Hypervisor Core： 以 AxVisor 解耦与多底座适配为例

FreeHypervisor 项目组

Abstract

Hypervisor 已经成为云计算、轻量级隔离、机密计算、嵌入式整合和安全关键系统中的基础软件组件。然而，现有 Hypervisor 往往与某一类宿主底座紧密绑定：Xen、ACRN、Bao、Jailhouse、NOVA 和 seL4 CAMkES VMM 等系统倾向于自建或依赖专用底座；KVM 和 bhyve 等 hosted hypervisor 则深度复用 Linux 或 FreeBSD 的内核机制。这两类路线分别证明了专用底座和通用 OS 复用的价值，但都没有充分回答一个问题：同一个 Hypervisor Core 能否跨不同宿主 OS 复用？

本文以 AxVisor 为对象，提出一种面向跨 OS 底座的可移植 Hypervisor 架构。我们将原本绑定 ArceOS 运行环境的 AxVisor 拆分为 OS-independent Hypervisor Core 与 OS-specific Host Adapter，并把宿主 OS 依赖收束为一组显式的 Host Capability Interface。当前实现中，这一接口边界落实为 30 个最小验证接口，覆盖任务执行、运行时安装、宿主 CPU 与 per-CPU 初始化、控制台、时间、同步、IRQ、内存分配、地址转换以及启动入口桥接等能力。进一步地，本文把完整系统边界组织为四层：host-api、runtime contract、hv-provider-api 与 device-model/control-plane，以避免把宿主基础设施、AxVisor 运行时、硬件虚拟化执行面和管理面混为一层。

我们在 ArceOS、Asterinas 和 Linux 三类差异明显的底座上实现或规划该接口。ArceOS 用于验证从原生运行环境中解耦后的兼容性；Asterinas 用于验证新型 Linux ABI-compatible OS 上的 host adapter 与 KVM-style control plane；Linux 则作为成熟通用内核验证 Host Capability Interface 是否能够映射到真实生产级 OS 机制。本文的核心结论是：Hypervisor 的可移植性不应只理解为硬件架构移植或 guest 支持，而应包含宿主 OS 运行时可移植性；这一目标需要显式的能力契约，而不是简单的 API 包装。

1 引言

虚拟化已经从传统服务器整合技术演化为构建现代系统的重要基础。云平台依赖 Hypervisor 实现多租户隔离；serverless 和容器场景使用轻量级虚拟机降低共享内核风险 [3, 2]；机密计算和基于 VM 的 TEE 依赖虚拟化边界构建可信执行环境；嵌入式、车载和工业系统则通过 Hypervisor 在同一硬件上整合通用 OS、实时 OS 与安全关键组件 [5, 1]。随着使用场景扩展，Hypervisor 不再只是“运行 guest OS 的工具”，而是调度、内存、I/O、中断、时间和管理面共同组成的系统级服务。

现有系统大致沿两条路线组织 Hypervisor 与宿主底座的关系。第一类系统自建或依赖专用底座。Xen 通过 dom0 和 toolstack 组织虚拟化系统；ACRN、Bao、Jailhouse 等面向嵌入式或静态分区场景，强调可预测性和资源隔离 [4, 7, 6]；NOVA 和 seL4 CAMkES VMM 则将虚拟化建立在微内核对象模型和组件框架之上。这类系统具有清晰边界和较强控制力，但需要维护一整套专门的系统栈。

第二类系统复用通用 OS。KVM 将 Linux 本身变成 Hypervisor runtime：VM 表现为进程，vCPU 由 Linux 调度，guest memory 关联用户态地址空间，控制面通过 /dev/kvm、fd、mmap 和 ioctl 暴露给 QEMU、Firecracker、Cloud Hypervisor 和 crosvm 等用户态 VMM。bhyve 在 FreeBSD 上采取类似 hosted hypervisor 路线。这条路线的优势是复用成熟 OS 的调度、内存、文件对象、事件通知和设备框架，显著降低工程成本。但其代价是虚拟化核心与具体宿主内核机制强耦合。

本文关注第三个维度：宿主 OS 可移植性。已有工作大量讨论 guest portability 和 hardware portability，例如支持不同 guest OS、x86/Arm/RISC-V 架构或不同硬件虚拟化扩

展 [8, 9]。但对于“Hypervisor Core 本身能否跨不同宿主 OS 复用”，现有系统给出的答案仍不充分。KVM 证明 Linux 可以成为优秀的 Hypervisor runtime，却没有将“Hypervisor 需要哪些宿主 OS 能力”提炼为可由其他 OS 实现的能力契约。

我们的核心观察是：Hypervisor 对宿主 OS 的依赖虽然语义复杂，但可以被组织成若干明确的能力域，包括任务执行、同步、时间、中断、内存分配、地址转换、平台初始化、运行时入口以及控制平面桥接。只要把这些依赖从隐式调用改写为显式 contract，Hypervisor Core 中相当一部分 VM/vCPU、guest memory、VM exit 和设备中介逻辑就不必绑定某一个 OS。

本文以 AxVisor 为例验证这一思路。AxVisor 原本运行在 ArceOS 生态中，直接依赖 ArceOS 的 axalloc、axhal、axtask、axruntime、per-CPU 机制、guard 体系以及 hypervisor mode 初始化逻辑。我们将这些依赖拆分为 host substrate、runtime substrate 和 native hypervisor substrate，并进一步用四层接口模型定义边界。当前实现将下层 host capability 收束为 30 个接口函数，作为 Linux、Asterinas 和 ArceOS 三个底座上的最小可验证边界。

本文贡献如下：

- 提出 Host Capability Model，把 Hypervisor 对宿主 OS runtime 的依赖显式化，区别于传统硬件 HAL。
- 将 AxVisor 解耦为 portable Hypervisor Core 与 Host Adapter，并以 30 个接口函数定义当前最小可验证边界。
- 提出四层接口框架：host-api、runtime contract、hv-provider-api、device-model/control-plane，明确宿主能力、运行时组织、硬件虚拟化执行面和管理面之间的职责分离。
- 在 ArceOS、Asterinas 和 Linux 三类底座上实现或规划适配，分析 Linux 与 Asterinas 在内存、IRQ、FDT、设备所有权等方面的语义差异。
- 将 KVM-compatible userspace ABI 分解为 control-plane frontend 与 AxVisor provider backend，为非 Linux OS 支持 Linux VMM workflow 提供结构化路径。

2 背景与动机

2.1 Hypervisor 对宿主 OS 的依赖

一个完整 Hypervisor 不仅需要执行 VM entry 和处理 VM exit，还需要组织一套运行时。至少包括以下能力：

- 执行与调度能力：承载 vCPU loop、管理 host task、处理 vCPU wakeup 和 yield。
- 内存能力：分配 host page、构造 guest memory backing、完成 host virtual/physical address translation。
- 时间能力：读取单调时间、设置 host timer、驱动 guest timer event。
- 中断能力：注册 host IRQ handler、处理 external interrupt、把事件转发到 guest interrupt injection 路径。
- 同步能力：维护 VM/vCPU 状态机、wait queue、条件等待和跨任务唤醒。
- 控制平面能力：向用户态 VMM 或管理工具暴露 VM 创建、vCPU 运行、内存注册和状态访问语义。

这些能力不是传统意义上的硬件抽象。它们抽象的是宿主 OS runtime 服务：任务、页框、wait queue、timer、IRQ、文件对象、mmap 对象和用户地址空间。本文称之为 Host Capability。

2.2 KVM 的启发与局限

KVM 的成功说明复用通用 OS 是构建 Hypervisor 的有效工程路线。Linux 已经提供调度器、内存管理、文件描述符、eventfd、poll/epoll、设备模型接入和用户态 ABI，因此 KVM 不必从零构建完整 Hypervisor OS。

但是，KVM 的宿主依赖并没有被抽象为独立接口。KVM 直接嵌入 Linux 的 task、mm、fd、ioctl、eventfd、GUP、RCU 和调度机制中。对于 ArceOS、Asterinas 或其他研究 OS

而言，复用 KVM 往往意味着同时引入大量 Linux 内核内部机制。这与“复用 Hypervisor Core”的目标并不相同。

本文继承 KVM “复用宿主 OS”的思想，但希望把它从 Linux 的具体实现中抽象出来：宿主 OS 不再是某个固定内核，而是一组显式 Host Capability 的提供者。

2.3 Linux ABI 兼容与虚拟化生态兼容的断层

近年来，许多新型 OS 通过 Linux ABI 兼容复用应用生态，例如 Asterinas、gVisor、Fuchsia Starnix、WSL、FreeBSD Linuxulator 和 illumos LX zones。这类工作主要解决普通 Linux 应用如何运行在非 Linux 内核上。

虚拟化应用更复杂。QEMU、Firecracker、Cloud Hypervisor、crosvm 和 lkvm 虽然运行在用户态，但它们依赖的不只是 `open/read/write/mmap/ioctl` 这些 syscall 表面形式，而是 `/dev/kvm` 背后定义的虚拟化对象语义：全局 KVM fd、VM fd、vCPU fd、`KVM_SET_USER_MEMORY_REGION`、`KVM_RUN`、vCPU run page、`ioeventfd` 和 `irqfd` 等。

因此，一个 OS 能运行普通 Linux 程序，并不意味着它能运行依赖 KVM 的 Linux VMM。本文将这称为 Linux application compatibility 与 Linux virtualization compatibility 之间的断层。

3 问题定义

本文围绕两个问题展开。

问题一：Hypervisor 的宿主依赖缺少可移植抽象。 现有系统要么自建底座，要么深度绑定通用 OS。我们希望回答：能否将 Hypervisor 对宿主 OS 的依赖提炼为显式 Host Capability Interface，使同一个 Hypervisor Core 运行在不同 OS 底座上？

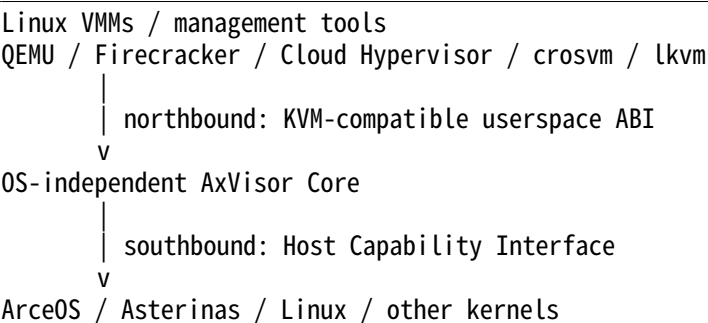
问题二：Linux VMM workflow 需要 KVM-compatible 虚拟化语义。 Linux ABI 兼容主要解决普通应用，而依赖 `/dev/kvm` 的 VMM 还需要 VM/vCPU/memory/run page/eventfd 等对象语义。我们希望回答：能否在 portable Hypervisor Core 之上提供 KVM-compatible frontend，使非 Linux OS 或替代 provider 支持部分 Linux VMM workflow？

问题一是本文主线，问题二是建立在问题一之上的生态兼容扩展。本文不试图替代 Linux KVM，也不声称完整实现所有 KVM ioctl、VFIO、nested virtualization、live migration 或生产级性能竞争。

4 总体设计

4.1 三层运行结构

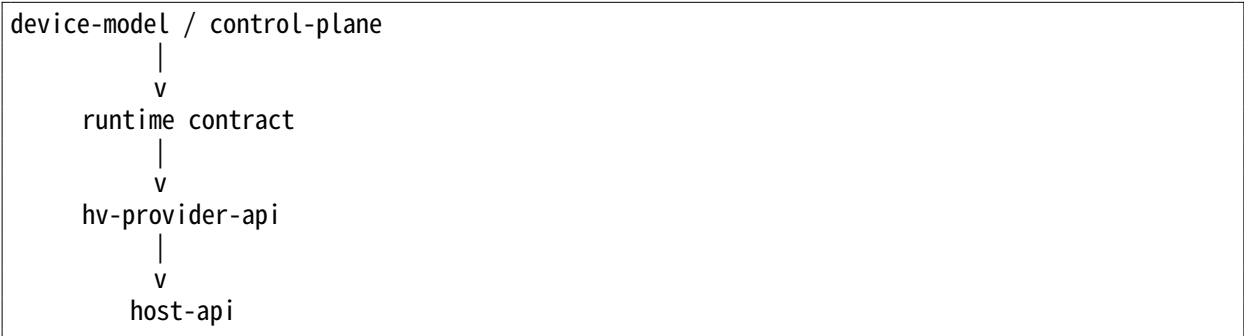
从运行路径看，系统可以抽象为三层：



下层 Host Capability Interface 描述 AxVisor Core 对宿主 OS runtime 的依赖；上层 KVM-compatible ABI 面向 Linux VMM workflow；AxVisor Core 位于中间，负责 VM/vCPU 生命周期、guest memory、VM exit、寄存器状态和虚拟化语义。

4.2 四层接口模型

为了避免把不同职责混在同一个 adapter 中，本文进一步把接口边界组织为四层：



host-api 提供宿主底座基础能力，例如内存分配、地址转换、任务、同步、时间、中断、平台探测和控制台。当前 30 个接口主要位于这一层，另有少量 runtime hook 和入口胶水。

hv-provider-api 表达真正的硬件虚拟化执行面，包括 VM/vCPU 创建、guest memory 注册、vCPU 运行、guest 状态访问、VM exit、虚拟中断注入和 guest 时间虚拟化。它依赖 host-api 的 building blocks，但不能被 host-api 取代。

runtime contract 负责 AxVisor 自身运行时组织，例如 VM/vCPU runtime 注册、vCPU loop、VM exit 分发、状态机、上下文协调和跨 vCPU 请求。

device-model/control-plane 负责配置、镜像装载、设备模型、管理接口以及 KVM-style frontend。QEMU/Firecracker 这类用户态 VMM 位于这一层之上或与这一层交互。

这四层划分的关键是职责分离。宿主内存分配不是 guest memory 注册；host task 不是 vCPU；host IRQ 不是 guest interrupt injection；host CSR/MSR 原语不是 guest register state；/dev/kvm fd/ioctl/mmap 也不是底层 hypervisor provider 本身。

5 Host Capability Interface

5.1 当前 30 个最小验证接口

当前实现将 AxVisor 对宿主底座的最小依赖收束为 30 个接口。它们不是最终接口全集，而是当前原型验证中用于划定 AxVisor Core 与 Host Adapter 边界的最小集合。

编号	接口	语义
1	KernelTaskRuntime::spawn_task	通过宿主执行体承载 AxVisor runtime 或 vCPU 相关任务。
2	install_kernel_task_runtime	安装宿主 task runtime，使 AxVisor runtime 能调用宿任务能力。
3	runtime 启动入口胶水	将宿主入口桥接到 AxVisor processor 或 boot path。
4	HostIf::get_host_cpu_num	查询宿主 CPU 数量。
5	HostIf::init_percpu	初始化 AxVisor 所需 per-CPU 状态。
6	HostIf::exit	从 AxVisor runtime 请求宿主退出或停止。
7	ConsoleIf::write_bytes	向宿主控制台输出字节流。
8	ConsoleIf::read_bytes	从宿主控制台或缓冲区读取输入。
9	TimeIf::current_time_nanos	读取宿主单调时间。
10	TimeIf::set_one-shot_timer	设置一次性宿主 timer，用于驱动 AxVisor timer event。
11	SyncIf::create_wait_queue	创建宿主 wait queue 对象。
12	SyncIf::destroy_wait_queue	销毁 wait queue 并处理生命周期。

编号	接口	语义
13	SyncIf::wait_queue_wait	当前任务在 wait queue 上等待。
14	SyncIf::wait_queue_wait_until	带条件或超时的等待。
15	SyncIf::wait_queue_wake_one	唤醒一个等待任务。
16	SyncIf::wait_queue_wake_all	唤醒全部等待任务。
17	TaskIf::spawn_task_raw	创建原始宿主任务。
18	TaskIf::join_task	等待宿主任务退出并回收状态。
19	TaskIf::current_task	获取当前宿主任务身份，用于 current vCPU context 等逻辑。
20	TaskIf::yield_now	主动让出 CPU。
21	IrqIf::handle_irq	处理宿主 IRQ 或 trap vector 并转发给 AxVisor。
22	IrqIf::register_irq_handler	注册宿主 IRQ handler。
23	MemoryIf::alloc_frame	分配单个宿主页框。
24	MemoryIf::alloc_contiguous_frames	分配物理连续宿主页框。
25	MemoryIf::dealloc_frame	释放单个宿主页框。
26	MemoryIf::dealloc_contiguous_frames	释放物理连续宿主页框。
27	MemoryIf::phys_to_virt	宿主物理地址到宿主虚拟地址转换。
28	MemoryIf::virt_to_phys	宿主虚拟地址到宿主物理地址转换。
29	runtime hook 安装链	安装 processor/runtime/timer/IRQ 相关 hook。
30	runtime 入口桥接链	将 Linux/Asterinas/ArceOS 入口接入真实 axvisor_core::boot::run()。

这组接口的意义不在于数量本身，而在于把原先散落在 ArceOS 内部的隐式依赖转化为显式 contract。它刻画的是 AxVisor Core 当前启动、运行和处理基础事件所需的宿主能力，而不是完整虚拟化 provider API。

5.2 接口状态

以 Linux adapter 为例，当前 30 个接口可以分为三类。

已经具备 Linux 语义封装的接口。 task/sync 相关接口已经映射到 Linux kthread、task registry、CondVar、Mutex 和完成通知；host/console/time 相关接口已经接入 CPU 查询、per-CPU 初始化、退出入口、控制台缓冲、单调时间和 RISC-V hrtimer backend；memory 相关接口已经接到 Linux 页分配、连续页分配和地址转换记录，不是简单 stub。

已有第一版但需要补强的接口。 runtime hook 和入口桥接链已经存在，并且 run() 路径能够通过 core_link::boot、boot_vendor_bridge、vendor::axvisor_core::boot 和 axvisor_linux_bridge_boot_run() 进入真实 axvisor_core::boot::run()。但是，这只能证明入口桥接成立，不能单独证明 Linux guest 启动协议完整匹配。IRQ 注册与分发也已经有骨架和 external IRQ event pipeline，但真实 IRQ 源号和 guest injection 语义仍需补强。

必须等真实 core 和真实 workload 接入后才能验证的接口。 TimeIf::set_one_shot_timer 已有 host timer backend，并能进入 axvisor_core::vmm::timer::check_events() 路径；但 timer event 是否满足 AxVisor Core 的真实语义，必须在真实 guest workload 下验证。类似地，Linux adapter 已经能识别、缓存和转发 external IRQ 事件，但是否能正确完成 guest interrupt injection，取决于 IRQ 源号、vCPU context 和 PLIC/APIC completion 语义。

6 实现

6.1 从 ArceOS 内嵌实现到 Portable AxVisor Core

AxVisor 原本依赖 ArceOS 的多个内部组件，包括 `axalloc`、`axhal`、`axtask`、`axruntime`、`axkspin guard` 体系、per-CPU 区、SBI firmware 对接、基础平台 I/O 以及 hypervisor mode 初始化逻辑。按真实运行内容看，这些依赖可以归为三类 substrate：

- host substrate：内存、时间、中断、架构寄存器、trap、地址空间、控制台和平台 I/O。
- runtime substrate：任务、guard、per-CPU、bootstrap、wait queue 和早期初始化顺序。
- native hypervisor substrate：VM/vCPU、world switch、G-stage/EPT/NPT、VM exit、guest CSR/MSR 状态、虚拟中断和虚拟 timer。

解耦后的 AxVisor Core 不再直接假设 ArceOS 的具体对象模型，而是通过 Host Adapter 获取宿主能力。ArceOS adapter 用于保持原生路径；Asterinas adapter 将 OSTD 和内核能力整理成 AxVisor 可用的 facade；Linux adapter 则通过 C/Rust bridge 将 Linux kernel objects 映射到同一接口。

6.2 ArceOS 适配

ArceOS 是 AxVisor 的原始运行环境。其价值在于验证：解耦之后的 AxVisor Core 仍能在原生底座上运行，而不是因为抽象化破坏既有语义。ArceOS 的 unikernel-style 结构使许多能力天然接近 AxVisor 需求，例如线性地址映射、直接平台初始化、轻量任务和静态 VM 配置。

在论文评估中，ArceOS static mode 应作为 southbound portability 的基线：宿主启动后直接进入 AxVisor，VM 配置和 guest 镜像在启动前准备，不依赖复杂用户态控制平面。

6.3 Asterinas 适配

Asterinas 代表一种 Linux ABI-compatible 但内核架构不同的新型 OS。Asterinas 适配承担两类验证。第一，它需要实现 Host Capability Interface，证明 AxVisor Core 可以脱离 ArceOS 私有假设，在另一种内核对象模型中运行。第二，它可以承载 KVM-style control plane，验证 Linux ABI 兼容能否进一步扩展到虚拟化应用 workflow。

Asterinas 侧最关键的收口点不是简单补齐 trait，而是明确底座语义：哪些地址可进行 `virt_to_phys`，哪些内存保证物理连续，哪些映射依赖 linear mapping，task/guard/per-CPU 语义如何与 AxVisor runtime 对齐。

6.4 Linux 适配

Linux adapter 的目标不是替代 KVM，而是验证 Host Capability Interface 是否能映射到成熟通用内核。Linux 与 ArceOS/Asterinas 的差异使它成为强验证底座：如果接口只能在研究 OS 或受控 unikernel 环境中成立，它的通用性有限；如果同一接口可以映射到 Linux 的 kthread、wait queue、timer、page allocator、memremap/ioremap 和 device release 语义，则更能说明 Host Capability Model 有跨 OS 意义。

当前 Linux adapter 已经具备以下基础：

- adapter 分层包括 `axvisor_adapter_main.rs`、`core_link/`、`vendor/`、`axvisor_core_stub.rs` 和 `arch/riscv64.rs`。
- 大多数 host capability 接口已有 Linux 语义封装，尤其是 task、sync、host、console、time 和 memory。
- vendor staging 已经把 AxVisor 主体源码和一批依赖源码放入 Linux 树内工作区。
- 真实 `axvisor_core::boot::run()` 入口桥接链已经成型，后续关键不再是补函数壳，而是打通 proc-macro、runtime crate、virtualization crate 和 `build.rs` 处理策略。

6.5 Linux 与 Asterinas 的关键语义差异

Linux 适配暴露出若干不能靠 API 包装掩盖的语义差异。

地址转换。 Asterinas 路径可以较自然地使用 `paddr_to_vaddr` 和反向线性映射。Linux loadable module 不能对任意 HPA 使用 `__va()`。当前 Linux `phys_to_virt` 需要区分 adapter-owned frames、连续分配、guest reserved RAM、passthrough MMIO、host PLIC MMIO 和不支持的任意物理地址。guest RAM 可能来自 `reserved-memory/no-map` 区域，需要 `memremap(MEMREMAP_WB)`；MMIO 和 PLIC 需要 `ioremap`。

guest RAM backing 与 DMA 可见性。 在 RISC-V passthrough 场景下，guest RAM 常采用 GPA=HPA 的 identical mapping，使 QEMU 或真实设备 DMA 能访问正确宿主物理地址。Linux host 需要在启动时把该区间从普通内存管理中排除，并保证 AxVisor 可通过受控映射访问。

外部 IRQ 注入上下文。 Asterinas 可以在处理 S external IRQ 时遍历 pending external interrupts 并调用 guest injection 路径。Linux 侧通常需要先 claim host PLIC IRQ、入队事件、唤醒或切入 vCPU 线程，再在合适的 vCPU context 中注入。若直接在非 vCPU IRQ context 中调用依赖 `current_vcpu_context` 的函数，可能无法完成注入。

passthrough 设备所有权。 Asterinas 不需要解绑 Linux platform driver。Linux 上如果要把 `virtio-mmio`、串口或其他 MMIO 设备交给 guest，必须先释放 host driver 所有权，否则 passthrough 不安全或不可用。

host FDT 来源。 Asterinas 可以由 boot path 暴露 FDT paddr。Linux module 无法可靠直接获取 boot DTB 物理地址，当前路径需要显式传入 `host_fdt_path` 或 `host_fdt_paddr`，并校验该 DTB 与 Linux host boot DTB 及 `reserved-memory` 节点一致。

7 KVM-compatible Control Plane

7.1 为什么它不是 30 个 host 接口的一部分

当前 30 个接口解决的是 AxVisor 如何在宿主内核运行，即 southbound host capability 问题。Firecracker、QEMU 或 Cloud Hypervisor 需要的是另一类接口：`/dev/kvm` 文件对象、KVM ioctl、vCPU fd `mmap(struct kvm_run)`、VM/vCPU fd 生命周期、userspace guest memory pinning、KVM_RUN 返回的 exit reason，以及 `ioeventfd/irqfd` 事件语义。

因此，KVM-compatible control plane 应作为 device-model/control-plane 层的一部分，映射到底层 AxVisor hv-provider-api，而不是塞进 host-api。

7.2 KVM ABI 对象模型

面向未修改 Firecracker 或其他 Linux VMM，需要提供如下对象模型：

```
/dev/kvm global fd
-> KVM_CREATE_VM returns VM fd
-> VM fd KVM_CREATE_VCPU returns vCPU fd
-> vCPU fd mmap exposes struct kvm_run
-> vCPU fd KVM_RUN enters guest
```

这一层应把 KVM ABI 分解为两部分：

- frontend 语义：ioctl number、fd 生命周期、struct `kvm_run` 布局、memory slot、eventfd、run page 和用户页 pin/unpin。
- backend 语义：创建 AxVisor VM/vCPU、注册 guest memory、运行 vCPU、读取/设置 guest 状态、把 AxVisor exit 翻译为 KVM exit reason。

例如，`KVM_SET_USER_MEMORY_REGION` 在 frontend 中负责接收 userspace HVA、slot、GPA 和 flags；在 Linux provider 中还要 pin userspace pages；随后通过 backend 把这些 host pages 注册到 AxVisor guest address space。KVM_RUN 则调用 AxVisor provider 的 `run_vcpu`，再把 `MmioRead/MmioWrite/IORead/IOWrite/Halt/SystemDown/FailEntry` 等 exit 翻译为 `KVM_EXIT_MMIO/KVM_EXIT_IO/KVM_EXIT_HLT/KVM_EXIT_SHUTDOWN` 或 `KVM_EXIT_FAIL_ENTRY`。

7.3 范围边界

第一阶段 KVM-compatible frontend 不应承诺完整 KVM。合理目标是支持一条确定的 VMM workflow，例如 x86_64 或 aarch64 Firecracker cold boot，并明确不支持 snapshot、migration、dirty logging、VFIO、nested virtualization 和完整 KVM capability parity。

对 Firecracker 而言，首阶段通常需要：

- 全局 fd: API version、capability check、vCPU mmap size 和 VM 创建，对应 KVM_GET_API_VERSION、KVM_CHECK_EXTENSION、KVM_GET_VCPU_MMAP_SIZE、KVM_CREATE_VM。
- x86_64 架构初始化: CPUID/MSR 查询，vCPU CPUID/MSR/REGS/SREGS/FPU 设置，以及 LAPIC/IRQCHIP/PIT 相关 ioctl。
- VM fd: guest memory region 注册、vCPU 创建、KVM_IOEVENTFD 和 KVM_IRQFD。
- vCPU fd: mmap run page、KVM_SET_SIGNAL_MASK、KVM_RUN。

8 评估计划

本文原型的评估应围绕接口充分性、可移植性、语义完整性和开销展开。

8.1 接口充分性

需要证明当前 30 个 Host Capability 接口足以支撑 AxVisor Core 的关键路径，包括启动 core、初始化 per-CPU、创建或恢复 VM 配置、运行 vCPU loop、处理 timer event、处理 external IRQ、管理 guest memory 和输出 guest console。

同时也要证明哪些语义不应纳入这 30 个接口，例如 VM/vCPU provider 生命周期、guest memory registration、guest register state 和 KVM fd/ioctl/mmap。这些应进入 hv-provider-api 或 control-plane 层。

8.2 代码复用率与适配成本

应统计 AxVisor Core 在 ArceOS、Asterinas 和 Linux 间共享的代码比例，以及各 Host Adapter 的代码规模。适配成本不应只按函数数量衡量，还应记录语义复杂度，例如 Linux phys_to_virt 地址分类、guest RAM reserved range、IRQ context 转换和设备 driver release。

8.3 多底座功能验证

推荐设置三类验证：

- ArceOS static mode: 验证解耦后原生路径仍能启动 guest。
- Asterinas static/control mode: 验证 Host Capability Interface 与 KVM-style workflow。
- Linux adapter mode: 验证真实 axvisor_core::boot::run() 入口、timer path、IRQ path、guest RAM backing 和 RISC-V 或 x86_64 guest smoke。

8.4 KVM-compatible ABI 覆盖

对于 KVM-compatible frontend，应列出当前支持的 ioctl 子集、对象生命周期、run page mmap、memory slot、MMIO pending read completion、ioeventfd 和 irqfd 状态。最终验收不应是“接口返回成功”，而应是未修改 VMM 的失败点按启动顺序推进，直至目标 guest 输出可复现 PASS 标志。

8.5 性能开销

性能评估不应声称超越 KVM，而应回答抽象层成本是否可接受。可测指标包括 vCPU entry/exit latency、MMIO exit 往返、timer event latency、IRQ injection latency、guest memory registration cost、adapter 调用层开销和 guest 启动时间。

9 相关工作

9.1 专用底座与 Type-I Hypervisor

Xen、ACRN、Bao 和 Jailhouse 代表自建或专用底座路线。Xen 通过 dom0 和 toolstack 管理设备与 VM；ACRN 面向 IoT 和嵌入式场景，通过 Service VM 组织设备和管理功能 [4]；Bao 和 Jailhouse 强调静态分区和混合关键性系统 [7, 6]。这类工作展示了专用 Hypervisor stack 的控制力，但通常需要维护完整底座和管理域。

9.2 微内核与高保证虚拟化

NOVA 和 seL4 CAMkES VMM 代表 microhypervisor 或微内核路线。它们关注 TCB 缩减、隔离性、组件化和形式化验证。但其虚拟化逻辑仍依赖特定微内核对象模型和组件运行时。本文不要求宿主 OS 采用特定微内核抽象，而是将宿主能力收束为 Hypervisor 所需的 runtime capability。

9.3 复用通用 OS 的 Hypervisor

KVM 和 bhyve 证明通用 OS 可以成为 Hypervisor runtime。KVM 深度复用 Linux 的调度、内存、fd、ioctl、eventfd 和用户态 ABI；bhyve 复用 FreeBSD 内核设施并通过 `/dev/vmm` 与用户态设备模型交互。本文继承 hosted hypervisor 的工程思想，但目标是把宿主依赖从 Linux/FreeBSD 内部机制中抽象出来。

9.4 轻量级 VMM 与 KVM 生态

Firecracker、Cloud Hypervisor、crosvm 和 QEMU 等用户态 VMM 构成 Linux KVM 生态。它们依赖 `/dev/kvm` 的对象语义，而不是仅依赖普通 syscall。本文的 KVM-compatible control plane 正是为了解耦这一层：让 KVM-style frontend 可以映射到 AxVisor provider，而不是要求直接移植 Linux KVM。

9.5 Linux ABI 兼容系统

Asterinas、gVisor、Fuchsia Starnix、WSL、FreeBSD Linuxulator 和 illumos LX zones 等系统从不同方向实现 Linux ABI 或 Linux-like environment。它们主要解决普通 Linux 应用兼容。本文进一步指出，虚拟化应用需要 KVM userspace ABI 的对象语义，因而 Linux virtualization compatibility 是比 syscall compatibility 更深的一层问题。

9.6 硬件虚拟化与 RISC-V

RISC-V H-extension、Arm virtualization extension 和 x86 VMX/SVM 相关工作主要关注硬件虚拟化机制和架构可移植性 [8, 9]。本文与之互补：硬件支持是 Hypervisor 的必要条件，但并不解决 Hypervisor Core 如何跨宿主 OS 复用的问题。

10 讨论与限制

本文工作有明确边界。第一，我们不试图在 Linux 上替代 KVM。AxVisor on Linux 的作用是验证 Host Capability Interface 在成熟 OS 上可实现，而不是竞争生产级 Linux 虚拟化栈。第二，当前 30 个接口是最小验证边界，不是最终完整接口全集。随着 KVM frontend、x86_64 backend、aarch64 backend、VFIO 或 migration 等能力加入，hv-provider-api 和 control-plane 层会继续扩展。第三，接口存在不等于语义成立。Linux 上尤其需要严格验证 guest RAM backing、phys_to_virt 地址类别、external IRQ injection context 和 passthrough device ownership。

此外，跨 OS 可移植性不是零成本。抽象层能减少重复实现 Hypervisor Core 的成本，但会把差异集中暴露在 Host Adapter 和 provider backend 中。本文主张的不是隐藏差异，而是让差异出现在正确的层次上。

11 结论

本文提出并实践了一种面向跨 OS 底座的可移植 Hypervisor 架构。通过将 AxVisor 拆分为 OS-independent Core 与 OS-specific Adapter，并用 30 个 Host Capability 接口定义当前最小验证边界，我们把原本隐式绑定 ArceOS 的运行时依赖转化为可由 ArceOS、Asterinas 和 Linux 分别实现的能力契约。

进一步地，本文用四层接口模型区分 host-api、runtime contract、hv-provider-api 和 device-model/control-plane，避免把宿主 OS 基础设施、AxVisor 运行时、硬件虚拟化执行面和 KVM-style 管理面混为一体。该结构说明：Hypervisor 可移植性不仅是硬件架构适配，也包括宿主 OS runtime 的可移植性；KVM 生态兼容也不仅是 syscall ABI，而是 /dev/kvm 对象语义与 provider backend 的组合。

当前实现已经证明大多数 host capability 可以在 Linux 上落到真实内核对象，并且 runtime 入口桥接已经进入真实 `axvisor_core::boot::run()` 路径。后续工作应继续完成真实 guest 启动闭环、严格化 Linux memory contract、修正 external IRQ 注入上下文，并推进 KVM-compatible frontend 到未修改 VMM 的可复现启动验证。

References

- [1] Alessandro Cilardo. Virtualization for embedded and cyber-physical systems: A survey. ACM Computing Surveys, 2022.
- [2] Tom Goethals, Bruno Volckaert, and Filip De Turck. A functional and performance evaluation of lightweight virtualization platforms for serverless computing. In Proceedings of the 2022 IEEE International Conference on Cloud Computing Technology and Science, 2022.
- [3] Connor Holmes, Laszlo Szekeres, and Dan R. K. Ports. Severifast: Lightweight virtualization for fast serverless computing. arXiv preprint, 2024.
- [4] Eddie Dong Li, Yaozu Xia, Baosheng Zang, and Haibing Guan. ACRN: A big little hypervisor for IoT development. In Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, pages 31–44. ACM, 2019.
- [5] Soledad Lozano, Benjamin Lesage, Laurent George, and Mohamed Abid. A comprehensive survey on hypervisors for embedded systems. ACM Computing Surveys, 2023.

- [6] José Martins, Adriano Tavares, and Sandro Pinto. Shedding light on static partitioning hypervisors for arm-based mixed-criticality systems. In Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium, 2023.
- [7] José Martins, Adriano Tavares, Marco Solieri, Marko Bertogna, and Sandro Pinto. Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems. In Workshop on Next Generation Real-Time Embedded Systems, 2020.
- [8] Bruno Sousa, José Martins, and Sandro Pinto. A first look at the risc-v virtualization extension. In Proceedings of the 18th ACM International Conference on Computing Frontiers, 2021.
- [9] Bruno Sousa, José Martins, and Sandro Pinto. HSPV: Hardware support for processor virtualization. ACM Computing Surveys, 2024.